

UNIT V

Branch and Bound Method: Least Cost (LC) search – the 15-puzzle problem – control abstractions for LC-Search – Bounding – FIFO Branch-and-Bound - 0/1 Knapsack problem – traveling salesman problem. Introduction to NP-Hard and NP-Completeness.

2 Marks

1. Define Branch-and-Bound method. (UQ APRIL'13 & APRIL'12)

The term Branch-and-Bound refers to all the state space methods in which all children of the E-node are generated before any other live node can become the E- node.

2. What are the searching techniques that are commonly used in Branch-and-Bound Method?

The searching techniques that are commonly used in Branch-and-Bound method are:

- i. FIFO
- ii. LIFO
- iii. LC
- iv. Heuristic search

3. What are NP- hard and Np-complete problems? (UQ NOV'12)

The problems whose solutions have computing times are bounded by polynomials of small degree.

4. What is a decision problem?

Any problem for which the answer is either zero or one is called decision problem.

5. What is maxclique problem?

A maxclique problem is the optimization problem that has to determine the size of a largest clique in Graph G where clique is the maximal sub graph of a graph.

6. What is approximate solution?

A feasible solution with value close to the value of an optimal solution is called approximate solution.

7. Explain Least Cost (LC) Search (UQ APRIL'13)

In LIFO Method, the selection (generation) of next E-node is a rigid one. The next E-node is selected in terms of level by level.

8. FIFO Branch and Bound (UQ NOV'12)

This problem begins with upper = ∞ as an upper bound on the cost of minimum-cost answer node., whenever the node is generated in the tree, $C'(x)$ and $u(x)$ are computed. Each time a new answer node is found, the value of upper can be updated by the value of $u(x)$

9. Differentiate backtracking and branch-and-bound. Techniques (UQ APR'11)

Backtracking Branch-and-bound

State-space tree is constructed using depth-first search State-space tree is constructed using best-first search Finds solutions for combinatorial non-optimization problems Finds solutions for combinatorial optimization problems.No bounds are associated with the nodes in the state-space tree Bounds are associated with the each and every node in the state-space tree

10. Compare the FIFO and LIFO search.

- BFS is an FIFO search in terms of live nodes
- List of live nodes is a queue
- DFS is an LIFO search in terms of live nodes

- List of live nodes is a stack

11. When can a search path be terminated in a branch-and-bound algorithm?

A search path at the current node in a state-space tree of a branch-and-bound algorithm can be terminated if

- The value of the node's bound is not better than the value of the best solution seen so far

The node represents no feasible solution because the constraints of the problem are already violated

- The subset of feasible solutions represented by the node consists of a single point in this case compare the value of the objective function for this feasible solution with that of the best solution seen so far and update the latter with the former if the new solution is better.

12. What is best-first branch-and-bound?

It is sensible to consider a node with the best bound as the most promising, although this does not preclude the possibility that an optimal solution will ultimately belong to a different branch of the state-space tree. This strategy is called best-first branch-and-bound.

13. What is knapsack problem?

Given n items of known weights w_i and values v_i , $i=1,2,\dots,n$, and a knapsack of capacity W , find the most valuable subset of the items that fit the knapsack. It is convenient to order the items of a given instance in descending order by their value-to-weight ratios. Then the first item gives the best payoff per weight unit and the last one gives the worst payoff per weight unit

14. What are the strengths of backtracking and branch-and-bound?

The strengths are as follows

- ✓ It is typically applied to difficult combinatorial problems for which no efficient algorithm for finding exact solution possibly exist
- ✓ It holds hope for solving some instances of nontrivial sizes in an acceptable amount of time
- ✓ Even if it does not eliminate any elements of a problem's state space and ends up generating all its elements, it provides a specific technique for doing so, which can be of some value.

15 Define SATISFIABILITY.

Let $x_1, x_2, x_3, \dots, x_n$ denotes Boolean variables.

Let x_i denotes the relation of x_i .

A literal is either a variable or its negation.

A formula in the propositional calculus is an expression that can be constructed using literals and the operators and $\wedge$ or $\vee$.

A clause is a formula with at least one positive literal. The satisfiability problem is to determine if a formula is true for some assignment of truth values to the variables.

16. What are the two classes of non polynomial time problems?

- NP? hard
- NP?complete

17. Define COOK'S theorem.

The satisfiability problem is to determine if a formula is true for some assignment of truth values to the variables. Satisfiability is in P if and only if $P = NP$.

18. Give the mathematical equation for 0/1 knapsack problem. (UQ APR'11)

A simple way to find the upper bound 'ub' is to add 'v', the total value of the items already selected, the product of the remaining capacity of the knapsack $W-w$ and the best per unit payoff among the remaining items, which is v_{i+1}/w_{i+1} $ub = v + (W-w)(v_{i+1}/w_{i+1})$

19.What is bounding function? (UQ:APRIL/MAY'14)

Bounding functions are used to kill live nodes without generating all their children.

20.what is non deterministic algorithm (UQ:APRIL/MAY'14/ APRIL 2015)

A conceptual *algorithm* with more than one allowed step at certain times and which always takes the right or best step. It is not random, as in *randomized algorithm*, or indeterminate. Rather it has the supercomputational characteristic of choosing the optimal behavior.

1. Explain Branch and bound method (UQ APRIL'12 & APRIL'13, APRIL/MAY'14)

The design technique known as **branch and bound** is very similar to backtracking (seen in unit 4) in that it searches a tree model of the solution space and is applicable to a wide variety of discrete combinatorial problems.

Each node in the combinatorial tree generated in the last Unit defines a *problem state*. All paths from the root to other nodes define the **state space** of the problem.

Solution states are those problem states's' for which the path from the root to's' defines a tuple in the solution space. The leaf nodes in the combinatorial tree are the solution states.

Answer states are those solution states's' for which the path from the root to's' defines a tuple that is a member of the set of solutions (i.e., it satisfies the implicit constraints) of the problem.

The tree organization of the solution space is referred to as the **state space tree**.

A node which has been generated and all of whose children have not yet been generated is called a **live node**.

The **live node** whose children are currently being generated is called the *E-node* (node being expanded).

A **dead node** is a generated node, which is not to be expanded further or all of whose children have been generated.

Bounding functions are used to kill live nodes without generating all their children.

Depth first node generation with bounding function is called backtracking. State generation methods in which the *E-node* remains the *E-node* until it is dead lead to **branch-and-bound method**.

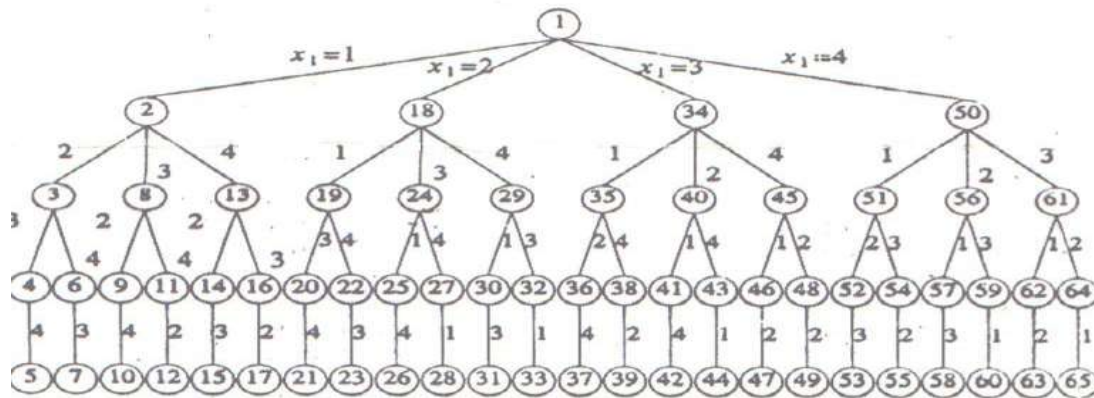
The term branch-and-bound refers to all state space search methods in which all children of the *E-node* are generated before any other live node can become the *E-node*.

In branch-and-bound terminology breadth first search(BFS)- like state space search will be called FIFO (First In First Output) search as the list of live nodes is a first -in-first -out list(or queue).

A **D-search** (depth search) state space search will be called LIFO (Last In First Out) search, as the list of live nodes is a list-in-first-out list (or stack). Bounding functions are used to help avoid the generation of sub trees that do not contain an answer node. The branch-and-bound algorithms search a tree model of the solution space to get the solution.

Example: 4-queens Problem.

Let us see how a FIFO branch-and-bound algorithm would search the state space tree for the 4-queens problem.



Initially, there is only one live node, node1. This represents the case in which no queen has been placed on the chessboard. This node becomes the E-node.

It is expanded and its children, nodes2, 18, 34 and 50 are generated.

These nodes represent a chessboard with queen1 in row 1 and columns 1, 2, 3, and 4 respectively.

The only live nodes are 2, 18, 34, and 50. If the nodes are generated in this order, then the next E-node is node 2.

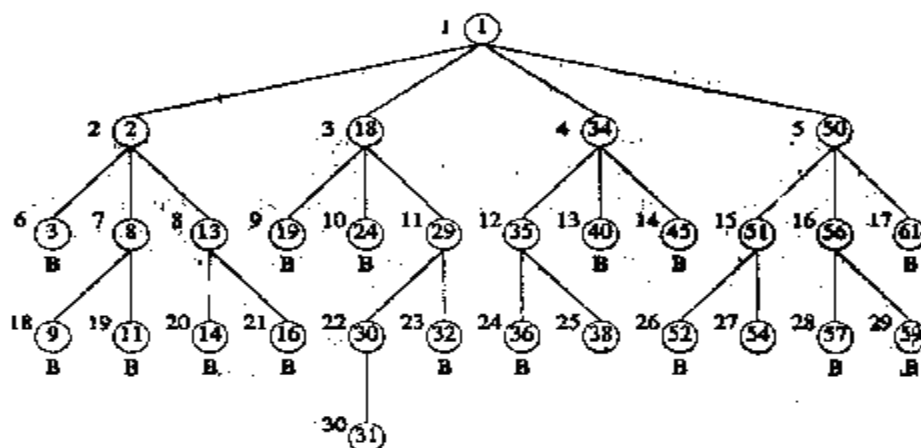
It is expanded and the nodes 3, 8, and 13 are generated. Node 3 is immediately killed using the bounding function. Nodes 8 and 13 are added to the queue of live nodes.

Node 18 becomes the next E-node. Nodes 19, 24, and 29 are generated. Nodes 19 and 24 are killed as a result of the bounding functions. Node 29 is added to the queue of live nodes.

Now the E-node is node 34. Figure 8.1 shows the portion of the tree of Figure 7.2 that is generated by a FIFO branch-and-bound search. Nodes that are killed as a result of the bounding functions are a "B" under them.

Numbers inside the nodes correspond to the numbers in Figure 7.2. Numbers outside the nodes give the order in which the nodes are generated by FIFO branch-and-bound.

At the time the answer node, node 31, is reached, the only live nodes remaining are nodes 38 and 54.



2. Explain Least Cost (LC) Search with 15 puzzle example(UQ APRIL'13)

- In LIFO Method, the selection (generation) of next E-node is a rigid one. The next E-node is selected in terms of level by level.
- The nodes in current level are expanded first and then go to the next level.
- This process is repeated until the nearest answer node is found
- In FIFO, it takes more time to search the nearest answer node, because of expansion of all nodes in each level
- Because of FIFO takes more time to search the nearest answer node we go for Least cost search(LC) method
- In LC search method, we can speed up the searching by ranking (cost) function.
- A search strategy that uses a cost function

$$C'(x) = f(h(x) + g'(x))$$

to select the next E-node (current live node is chosen as next E-node) with least Cost. Hence such a strategy is called LC search

$h(x)$ – cost of reaching x from root (no of levels from root to any node x)

$g(x)$ -cost of reaching a nearest answer node(answer node with minimum cost) from any node x (no of levels from any node x to a nearest answer node)

- Assign the rank (cost) to the node requires
 - Computation of cost needed to search an answer node from the live
 - For any node x , the cost is the number of levels the nearest answer node is from x .(that is number of levels from x to the nearest answer node)

Control abstraction for LC search

Let

t - state space tree

$c()$ - cost function for the nodes in t

if x is a node in t , then $c(x)$ is the minimum cost of any answer node in the sub tree with root x

Thus, $c(t)$ – cost of a minimum cost answer node in t

We cannot find $c(.)$ easily, so heuristic $C'()$ is used

listnode = record

```
{
listnode *next, *parent;
Float cost;
}
```

Algorithm LC Search(t)

\ Search t for an answer node

```
{
  If *t is an answer node then output *t and return;
  E = t ; \ E-node
  Initialize the list of live nodes to be empty;
  Repeat
  {
    For each child  $x$  of E do
    {
      If  $x$  is an answer node the output the path
```

```

        from x to t and return
        Add(x) ; \ x is a new live node.
        (x->parent):=E; \Pointer for path to root.
    }
    If there are no more live nodes then
    {
        Write("No answer node"); return;
    }
    E=Least();
} until (false);
}

```

- The LC Search algorithm uses c' to find an answer node
- Two function Add(x) and Least() are used
 - Add(x) -> Adds new live node x to the list of live nodes
 - Least() -> finds a live node with least c'(). This node is deleted from the list of live nodes and returned.
- It outputs the path from the answer node it finds to the root node t
 - If each x becomes live node, then associate a field parent which gives the parent of node x
 - When answer node g is found, the path from g to t can be determined.
 - g - answer node
 - x - live node
 - t - root node

Algorithm explanation:-

- 1) Variable E point the current E-node
- 2) Initially root node t is the first E-node ie E = t ;
- 3) Initially the list of live nodes are empty
- 4) For loop examines all the children of the E-node
 - a. If x (one of the children) is answer node then output the path from x to t and exit
 - b. If x is not a answer node then x becomes a live node and add it to the list of live nodes. Its parent pointer is set to E
- 5) When all the children are generated, E-node becomes a dead node
- 6) If there are no more live nodes then print no answer node and exit else call Least() to choose the next E-node
- 7) repeat the steps 4 and 6 till if any answer node is found or if entire space tree has been searched

Example

15 Puzzle Problems

- the 15 puzzle consists of 15 numbered tiles on a square frame with a capacity of 16 tiles
- Initial arrangement of the tiles are given, the objective of the problem is to transform this arrangement into the goal arrangement through a series of legal moves.
- The legal moves are ones in which a tile adjacent to the empty spot is moved to Empty Spot

1	2	3	4
---	---	---	---

5	6	7	8
9	10	11	12
13	14	15	

- From the initial arrangement, four moves are possible
- We can move any one of the tiles numbered 2,3,5,6 to the empty spot
- These arrangements are called states of the puzzle
- To speed up the search, we can associate the cost $c(x)$ with each node x in the state space tree.
- $C'(x) = f(x) + g'(x)$ where $f(x)$ – length of the path from root to node x
 $g'(x)$ – an estimate of the length of a shortest path from x to a goal node in the subtree with root x

(or)

$g'(x)$ – number of nonblank tiles not in their goal position

1	2	3	4
5	6	7	8
9	10		12
13	14	15	11

- An LC search start with root node (node 1) as E-node
- The E-node node 1 generates all its children 2,3, 4 and 5
- After E-node node 1 has generated all its children it becomes dead node
- The live nodes are 2,3 4 and 5
- To determine the next E-node, $C'(x)$ is computed for all live nodes and the live node with least $C'(x)$ is selected as next E-node

$$C'(x) = f(x) + g'(x)$$

$$C'(2) = 1 + 4 = 5$$

$$C'(3) = 1 + 4 = 5$$

$$C'(4) = 1 + 2 = 3 \text{ (least cost node)}$$

$$C'(5) = 1 + 4 = 5$$

- The live node with least C' is node 4, which becomes the next E-node. It generates all its children 10,11 and becomes dead node
- The live nodes are 2,3,5,10,11 and 12
- To determine the next E-node, $C'(x)$ is computed for all live nodes and the live node with least $C'(x)$ is selected as next E-node

$$C'(x) = f(x) + g'(x)$$

$$C'(2) = 1 + 4 = 5$$

$$C'(3) = 1 + 4 = 5$$

$$C'(5) = 1 + 4 = 5$$

$$C'(10) = 2 + 1 = 3 \text{ (least cost node)}$$

$$C'(11) = 2 + 3 = 5$$

- The live node with least C' is node10, which becomes the next E-node. It generates all its children 22 and 23 and becomes dead node
- The node 23 is determined as goal node and the search terminates

In this case LC search is as efficient as using exact function $C'()$

The LC search will be far more selective than any other search methods

3. Explain Bounding function

- A branch -and-bound searches the state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node.
- We assume that each answer node x has a cost $c(x)$ associated with it and that a minimum-cost answer node is to be found. Three common search strategies are FIFO, LIFO, and LC.
- A cost function $\hat{c}()$ such that $\hat{c}(x) \leq c(x)$ is used to provide lower bounds on solutions obtainable from any node x .
- If **upper** is an upper bound, then all live nodes x with $\hat{c}(x) > \text{upper}$ may be killed as all answer nodes reachable from x have cost $c(x) \geq \hat{c}(x) > \text{upper}$
- The starting value for upper can be set to infinity.
- Each time a new answer node is found, the value of upper can be updated
- $C'(x)$ and upper and $c(x)$ can be computed using formulas

Example

Job Sequencing Problem

Given n jobs and one processor

Each job i is defined as 3 tuples (P_i, d_i, t_i)

t_i - processing time in units

d_i - deadline

P_i - penalty, if the job i is not completed within the deadline, then penalty P_i is incurred

Objective of the problem

To select the subset J of n jobs such that all jobs in J can be completed by their deadlines. Hence a penalty is incurred only on those jobs not in J . The subset J should have minimum penalty among all possible subsets J . such that a J is optimal.

Let us assume that $n=4$

$(P_1, d_1, t_1) = (5, 1, 1)$

$(P_2, d_2, t_2) = (10, 3, 2)$

$(P_3, d_3, t_3) = (6, 2, 1)$

$(P_4, d_4, t_4) = (3, 1, 1)$

- solution space for this instance consist of all possible subsets of the job index set $\{1, 2, 3, 4\}$
job index : 1, 2, 3, 4
solution space : all possible subset of job index
ie $\{1, 2, 3\}$ $\{2, 3\}$ $\{1, 4\}$ $\{1, 3, 4\}$,.....
- solution space can be organized into a tree by means of two formulations
 1. variable tuple size formulation
 2. fixes tuple size formulation

Variable tuple size formulation



Node – answer node

☐ Node - infeasible subsets

Compute $c(x)$ using the following rule

if x is a leaf node,

$c(x)$ – summation of penalty of the jobs which are not in the path from root node to any node x

if x is a root node

$c(x)$ - minimum penalty corresponding to any node in the subtree within root x
(or)

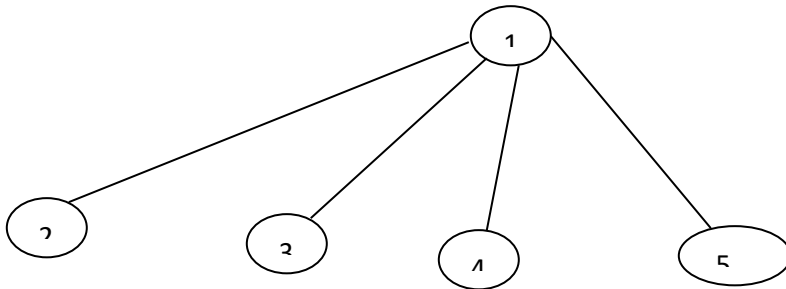
$c(x) = \min\{c(\text{child-1}), c(\text{child-2}), \dots, c(\text{child-n})\}$, where x is node in subtree

$c(x) = \infty$ for ☐ node

1. FIFO Branch and Bound

- This problem begins with upper = ∞ as an upper bound on the cost of minimum-cost answer node.
- Whenever the node is generated in the tree, $C'(x)$ and $u(x)$ are computed.
- Each time a new answer node is found, the value of upper can be updated by the value of $u(x)$
- For any node, if $c'(x) > \text{upper}$ then the node will be killed

Start with node1 as E-node and generate all its children 2, 3, 4 and 5. So compute $C(2), C'(3), C'(4), C'(x)$ and $u(2), u(3), u(4), u(5)$



Computation of $c'(x)$ using

$$c'(x) = \sum_{\substack{i < m \\ i \text{ is not } \in S_x}} P_i$$

Where x denotes nodes

S_x – number of given jobs

That is $S_x = \{1, 2, 3, 4\}$

Compute $c'(x)$ for node 1 ($x=1$)

- Initially the set S_1 is empty set because jobs are not selected into the set
- $S_1 = \{ \}$
- $m=0$ (m is the maximum element in the set)
- To determine the value for P , refer the set S_x and test the elements which should not exist in s_1 and it should be less than m
- no element in the set S_x satisfies both the condition, so $P = 0$
- $c'(1) = P_0 = 0$ (P_0 (penalty of zeroth job) is not given in the tuple)

Compute $c'(x)$ for node2 ($x=2$)

- if the first job only is included into the set,

- $S_2 = \{ 1 \}$
- $m = 1$ (m is the maximum element in the set)
- To determine the value for P_1 , refer the set S_x and test the elements which should not exist in s_2 and it should be less than m
- no element in the set S_x satisfies both the condition, so $P=0$
- $c'(2)=P_0=0$

Compute $C'(x)$ for node3 ($x=3$)

- if the second only job is included into the set,
- $x=3$ (node2) $S_3 = \{ 2 \}$
- $m = 2$ (maximum element in the set S_2)
- Refer S_x and determine $P=1$ (which is less than m and not exist in S_3)
- $c'(3)=P_1 = 5$

Compute $C'(x)$ for node4 ($x=4$)

- if the Third job only is included into the set,
- $S_4 = \{3\}$ $m=3$ (maximum element in the set S_4)
- Refer S_x and determine $P=1,2$ (which are less than m and not exist in S_4)
- $c'(4)= P_1+P_2=5+10= 15$

Compute $C'(x)$ for node5 ($x=5$)

- if 4 th job only is included into the set,
- $S_5 = \{4\}$ $m=4$ (maximum element in the set S_5)
- Refer S_x and determine $P=1,2,3$ (which are less than m and not exist in S_5)
- $c'(5)=P_1+P_2+P_3=5+10+6=21$

Computation of $u(x)$ using $u(x) = \sum_{i \text{ is not } \in S_x} P_i$

$u(x)$ for node 1($x=1$)

$S_1 = \{ \}$

$u(1) = \infty$ (assume that u value of root node and upper $= \infty$)

Initially upper $= u(1) = \infty$

$u(x)$ for node 2

$S_2 = \{1\}$

Refer S_x and determine $P=2,3,4$ (which are not exist in S_2)

$u(2)=P_2+P_3+P_4=10+6+3=19$

$u(x)$ for node 3

$S_3 = \{2\}$

Refer S_x and determine $P=1,3,4$ (which are not exist in S_3)

$u(3)=P_2+P_3+P_4=5+6+3=14$

$u(x)$ for node 4

$S_4 = \{3\}$

Refer S_x and determine $P=1,2,4$ (which are not exist in S_4)

$u(4)=P_1+P_2+P_4=5+10+3=18$

$u(x)$ for node 5

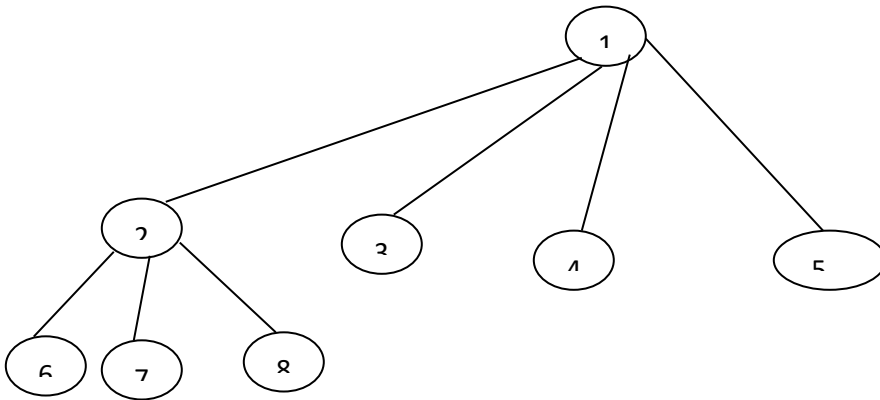
$S_5 = \{4\}$

Refer S_x and determine $P=1,2,3$ (which are not exist in S_5)

$u(5)=P_1+P_2+P_3=5+10+6=21$

- upper $= \min\{u(x) \text{ value of all nodes in current level}\}$ upper $= \min\{u(2), u(3), u(4), u(5)\}$
- upper $= \min(19, 14, 18, 21) = 14$
- Since $C'(4)$ and $C'(5)$ are greater than upper, the node 4 and 5 are killed (or Bounded)

- Only the nodes 2 and 3 remain alive.
- Node 2 becomes the next E-node
- Its children **nodes 6,7 and 8** are generated. So, Compute $C'(6), C'(7)$, $C'(8)$ and $u(6), u(7), u(8)$



Computation of $c'(x)$

Compute $C'(x)$ for node6 ($x=6$)

if job 1 and 2 are included into the set,

$S_6 = \{1, 2\}$ $m=2$ (maximum element in the set S_6)

Refer S_x and determine $P=0$ (no element in S_x satisfy the rule (which are less than m and not exist in S_6))

$$C'(6)=P_0=0$$

Compute $C'(x)$ for node7 ($x=7$)

if job 1 and 3 are included into the set,

$S_7 = \{1, 3\}$ $m=3$ (maximum element in the set S_7)

Refer S_x and determine $P=2$ (which is less than m and not exist in S_7)

$$C'(7)=P_2=10$$

Compute $C'(x)$ for node3 ($x=3$)

$S_8 = \{1, 4\}$ and $m=4$ (maximum element in the set S_8)

Refer S_x and determine $P=2, 3$ (which are less than m and not exist in S_8)

$$C'(8)=P_2+P_3=10+6=16$$

Computation of $u(x)$

$u(x)$ for node 6($x=6$)

$S_6 = \{1, 2\}$

Refer S_x and determine $P=3, 4$ (which are not exist in S_6)

$$u(6)=P_3+P_4=6+3=9$$

$u(x)$ for node 7

$S_7 = \{1, 3\}$

Refer S_x and determine $P=2, 4$ (which are not exist in S_7)

$$u(7)=P_2+P_4=10+3=13$$

$u(x)$ for node 8

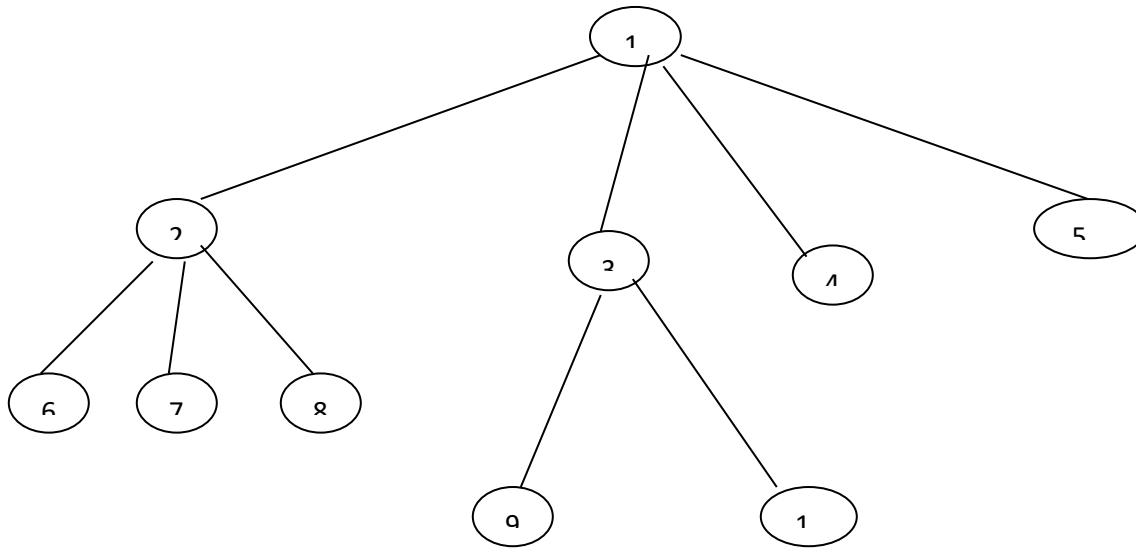
$S_8 = \{1, 4\}$

Refer S_x and determine $P=2, 3$ (which are not exist in S_8)

$$u(8)=P_2+P_3=10+6=16$$

- Upper= $\min\{u(x) \text{ value of all nodes in current level}\}$ upper= $\min\{u(6), u(7), u(8)\}$
- Upper= $\min(9, 13, 16)=9$

- The cost $C'(7)=10 > \text{upper}$ so, the node 7 gets killed
- Node 8 is infeasible (if the jobs chosen from root node to any node x (that is $x=8$ here) cannot be completed within its deadline which is called infeasible) and so it is killed
- Next, node 3 becomes E-node and its children nodes 9 and 10 are generated. So Compute $C'(9), C'(10)$ and $u(9), u(10)$



Computation of $c'(x)$

Compute $C'(x)$ for node 9 ($x=9$)

$S_9 = \{2, 3\}$ and $m=3$ (maximum element in the set S_9)

Refer S_x and determine $P=1$ (which is less than m and not exist in S_9)

$$C'(9) = P_1 = 5$$

Compute $C'(x)$ for node 10 ($x=10$)

$S_{10} = \{2, 4\}$ and $m=4$ (maximum element in the set S_{10})

Refer S_x and determine $P=1, 3$ (which are less than m and not exist in S_{10})

$$C'(10) = P_1 + P_3 = 5 + 6 = 11$$

Computation of $u(x)$

$u(x)$ for node 9 ($x=9$)

$S_6 = \{2, 3\}$

Refer S_x and determine $P=1, 4$ (which are not exist in S_9)

$$u(9) = P_1 + P_4 = 5 + 3 = 8$$

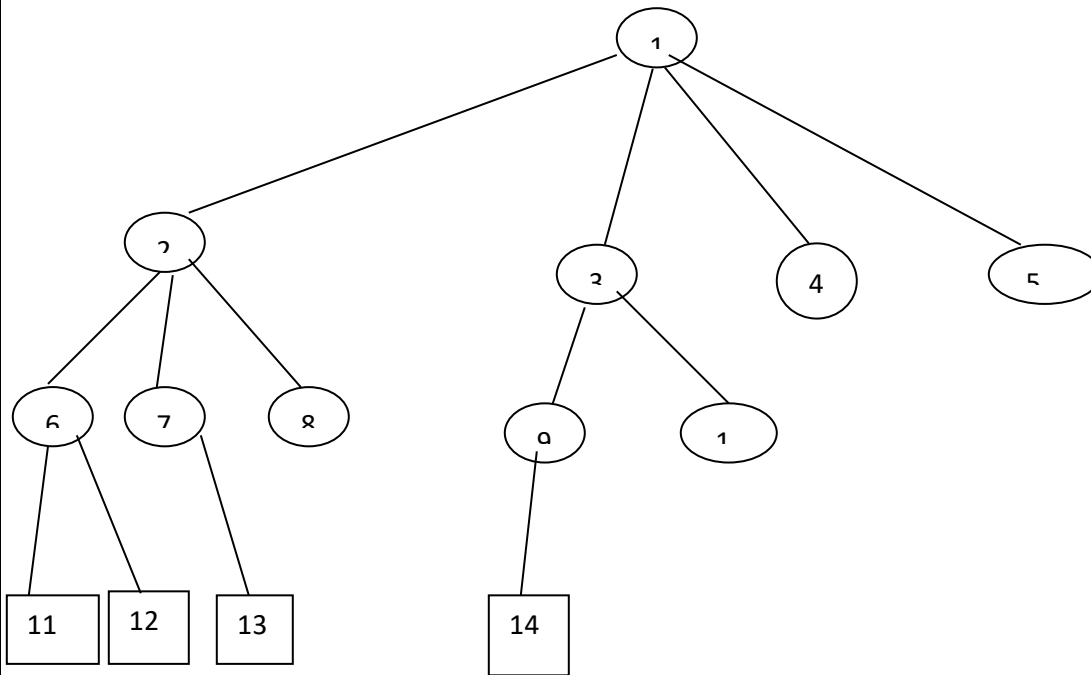
$u(x)$ for node 10 ($x=10$)

$S_7 = \{2, 4\}$

Refer S_x and determine $P=1, 3$ (which are not exist in S_{10})

$$u(10) = P_1 + P_3 = 5 + 6 = 11$$

- Upper = $\min\{u(x) \text{ value of all nodes in current level}\}$ upper = $\min\{u(9), u(10)\}$
- Upper = $\min(8, 11) = 8$. (node 9)
- The cost $C'(10) = 11 > \text{upper}$ so, it is killed
- The next E-node is 6. Both its children are infeasible and so killed
- Node 9 becomes the next E-node and its only child is also infeasible so, it is killed
- No live nodes remain so, the search terminates with node 9 representing minimum –cost answer node. It has a cost of 8



Computation C(x)

if x is a leaf node,

$c(x)$ – summation of penalty of the jobs which are not in the path from root node to any node x

$x=4, 5, 6, 7, 8, 9, 10$

$C(4)$ (Jobs 1,2 and 4 are not chosen from root node 1 to leaf node 6) = $P_1+P_2+P_4=5+6+3=14$

$C(5)$ (Jobs 1,2 and 3 are not chosen from root node 1 to leaf node 6)

= $P_1+ P_2+P_3=5+10+6=19$

$C(6)$ (Jobs 3 and 4 are not chosen from root node 1 to leaf node 6) = $P_3+P_4=6+3=9$

$C(7)$ (Jobs 2 and 4 are not chosen from root node 1 to leaf node 6) = $P_2+P_4=10+3=13$

$C(8)$ (Jobs 2 and 3 are not chosen from root node 1 to leaf node 6) = $P_2+P_3=10+6=16$

$C(9)$ (Jobs 1 and 4 are not chosen from root node 1 to leaf node 6) = $P_1+P_4=5+3=8$

$C(10)$ (Jobs 1 and 3 are not chosen from root node 1 to leaf node 6) = $P_1+P_3=5+6=11$

if x is a root node

$c(x)$ - minimum penalty corresponding to any node in the subtree within root x

(or)

$c(x)=\min\{c(\text{child-1}),c(\text{child-2}),\dots,c(\text{child-n})\}$,where x is any root node in subtree

$x=2,3$ (4,5 are killed so it does not have any child) and 1

$C(2) = \min\{ C(6),C(7),C(8)\}$

= $\min\{9,13,16\}$

= 9

$C(3) = \min\{ C(9),C(10)\}$

= $\min\{8,11\}$

= 8

$C(1) = \min\{C(2),C(3),C(4),C(5)\}$

$$= \min\{9,8,14,19\}$$

$$= 8$$

4. Explain Knapsack problem (UQ NOV'10)

- To use branch and bound technique to solve any problem, it is first necessary to conceive of a state space tree for the problem.
- Branch and bound technique is used to solve minimization problem(eg 15 puzzle, 4 queen, node with minimum cost is selection criteria)
- The branch and bound technique can not be applied directly in the knapsack problem, because the knapsack problem is a maximization problem(eg maximum profit is the selection criteria)
- This difficulty is easily overcome by replacing the objective function $\sum p_i x_i$ by the function $-\sum p_i x_i$
- The modified knapsack problem is stated as

$$\begin{aligned} \text{Minimize} \quad & - \sum_{i=1}^n p_i x_i \\ \text{subject to} \quad & \sum_{i=1}^n w_i x_i \leq m \\ & x_i = 0 \text{ or } 1, \quad 1 \leq i \leq n \end{aligned}$$

$$x_i = 0 \text{ or } 1, \quad 1 \leq i \leq n$$

Two types of formulation

Fixed tuple size formulation
Variable tuple size formulation

Fixed tuple size formulation

- Every leaf node in the state space tree representing an assignment for which

$$\sum_{1 \leq i \leq n} w_i x_i \leq m \text{ is an answer node}$$

- all other leaf nodes are infeasible
- for minimum cost answer node to correspond to any optimal solution, we need to define $c(x) = - \sum_{1 \leq i \leq n} p_i x_i$ for every answer node x
- the cost $c(x) = \alpha$ for infeasible leaf nodes

Example

Consider the knapsack instance $n=4, (p_1, p_2, p_3, p_4) = (10, 20, 12, 18), (w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$, and $m=15$

- there are two methods to solve the knapsack problem
 - LC Branch and Bound Solution
 - FIFO Branch and Bound Solution

FIFO Branch and Bound Solution

- The search begins with root as the E-node. That is node1(root) becomes E-node

Algorithm Ubound(cp,cw,k,m)

// cp is the current profit total ,cw is the current weight total; k is the index of the last removed item; and m is the knapsack size//

```
{
    b=cp; c=cw;
    for I=k+1 to n do
    {
        if((c+w[I])<=m) then
        {
            c=c+w[I]; b=b-p[I];
        }
    }
    return b;
}
```

Algorithm explanation with example

Ubound(0,0,0,15)

b=0; c=0;
for i= 1 to 4

i=1;
if ((c+w[1])<=m) (c=0 and w[1]=2 and 0+2=>2<=15) **true**

c=0+2= 2
b=0-10= -10

i=2
if ((c+w[2])<=m) (c=2 and w[2]=4 and 2+4=6<=15) **true**

c=2+4=6
b=-10-10= -20

i=3
if ((6+w[3])<=m) (c=6 and w[2]=6 and 6+6=12<=15) **true**

c=6+6=12
b=-20-12= -32

i=4

if ((15+w[4])<=m) (c=12 and w[2]=9 and 12+9=21<=15) **False**

return b(b=-32); (root node)=b

- for any node x in the state space tree, we compute c'(x) and u(x) and upper
- for any node x , if c'(x)>upper the node x is killed immediately
- initially the upper =u(1) and then the upper is updated by the node with minimum u(x) value in current level

u() for root node is calculated using the algorithm Ubound or (-Σ fully included objects

u(1)=b(return value of UBound algorithm)=-32 (objects1, 2 and 3 are fully includes so -Σ p1+p2+p3 =-(10+10+12)=-32)

Node 1

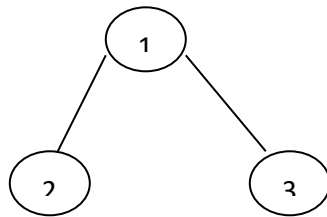
Initially, first 3 whole objects and part of 4 th object are included in the knapsack, that is (1,1,1,3/9)

$$c'(x) = - \sum_{1 \leq i \leq n} p_i x_i$$

$$c'(1) = -1*10 + 1*10 + 1*12 + 3/9*18 = -32$$

upper = u(1) = -32 (- $\sum$ fully (not a part of object) included objects (that is 1, 2 and 3) - $\sum p_1 + p_2 + p_3 = -(10+10+12) = -32$)

- E-node node 1 generates all its children 2,3 and becomes dead node. Now node 2 and 3 are live nodes.
- To determine the next E-node compute $c'(x)$, $u(x)$ and upper for live nodes 2 and 3



Node 2

If first object is included (already the first object is included in node 1 so same value is obtained for $c'(2)$) status of Included objects; (1,1,1,3/9)

$$c'(2) = -1*10 + 1*10 + 1*12 + 3/9*18 = -38$$

$$u(1) = -32 (- \sum \text{fully included objects (that is 1, 2 and 3)} - \sum p_1 + p_2 + p_3 = -(10+10+12) = -32)$$

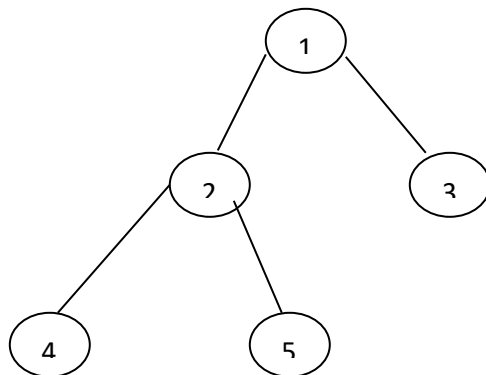
Node 3

If first object is not included , status of Included(objects; (0,1,1,5/9)

$$c'(3) = -0*10 + 1*10 + 1*12 + 5/9*18 = -32$$

$$u(3) = -22 (- \sum \text{fully included objects (that is 2 and 3)} - \sum p_2 + p_3 = -(10+12) = -22)$$

- upper = min{ $u(2)$, $u(3)$ }
= min{ -32, -22 }
upper = -32
- if $c'(2)$ and $c'(3)$ < upper, so node 2 and 3 are not killed and are live nodes
- The node 3 becomes the next E-node because $c'(2) = -38 < c'(3) = -32$
- Node 3 generates all its children 4, 5



Node 4

If second object is included (already the third object is included in node 1, 2 so same value is obtained for $c'(4)$) status of Included(objects; (1,1,1,3/9)

$$c'(4) = -1*10 + 1*10 + 1*12 + 3/9*18$$

$$= -38$$

$$u(4) = -32(-\sum \text{fully included objects (that is 1, 2 and 3)} - \sum p_1 + p_2 + p_3 = -(10+10+12) = -32)$$

Node 5

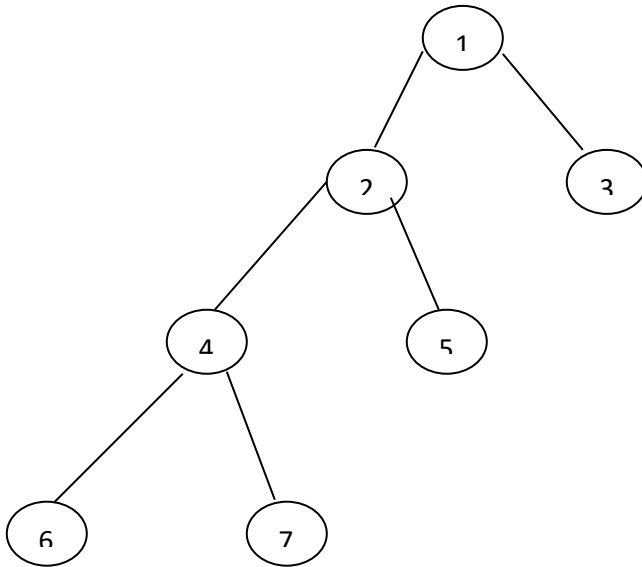
If second object is not included, status of Included objects; (1, 0, 1, 7/9)

$$c'(5) = -1*10 + 0*10 + 1*12 + 7/9*18$$

$$= -36$$

$$u(5) = -22(-\sum \text{fully included objects (that is 1 and 3)} - \sum p_1 + p_3 = -(10+12) = -22)$$

- $\text{upper} = \min\{u(4), u(5)\}$
 $= \min\{-32, -22\}$
 $\text{upper} = -32$
- if $c'(4)$ and $c'(5) < \text{upper}$, so node 4 and 5 are not killed and are live nodes
- Within the live nodes 4 and 5, the node 4 becomes the next E-node because $c'(4) = -38 < c'(5) = -36$
- Node 4 generates all its children 6, 7



Node 6

If third object is included (already the third object is included in node 1, 2, 4, so same value is obtained for $c'(6)$) status of Included objects; (1, 1, 1, 3/9)

$$c'(6) = -1*10 + 1*10 + 1*12 + 3/9*18$$

$$= -38$$

$$u(6) = -32(-\sum \text{fully included objects (that is 1, 2 and 3)} - \sum p_1 + p_2 + p_3 = -(10+10+12) = -32)$$

Node 7

If third object is not included, status of Included objects; (1, 1, 0, 1)

$$c'(7) = -1*10 + 1*10 + 0*12 + 1*18$$

$$= -38$$

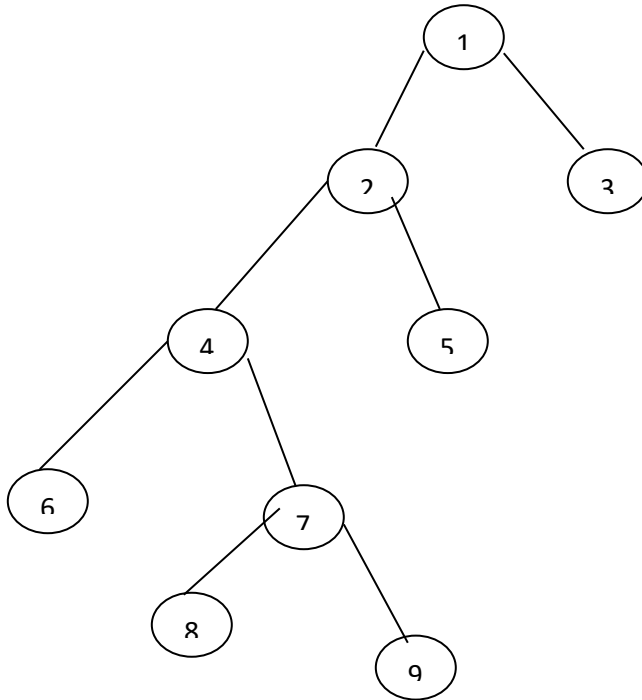
$$u(7) = -38(-\sum \text{fully included objects (that is 1, 2 and 4)} - \sum p_1 + p_2 + p_4 = -(10+10+18) = -38)$$

- $\text{upper} = \min\{u(6), u(7)\}$

$$= \min\{-32, -38\}$$

$$\text{upper} = -38$$

- if $c'(6)$ and $c'(7)$ = upper, so node 4 and 5 are not killed and are live nodes
- Within the live nodes 6 and 7, the node 7 becomes the next E-node because $u(7) = -38 < u(6) = -32$ ($c'(6) = -38$ is equal to $c'(7) = -36$ so compare $u(x)$)
- Node 7 generates all its children 8, 9



Node 8

If fourth object is included status of Included objects; (1,1,0,1)

$$c'(6) = -1*10 + 1*10 + 0*12 + 1*18$$

$$= -38$$

$$u(8) = -38 (-\sum \text{fully included objects (that is 1, 2 and 4)} - \sum p_1 + p_2 + p_4 = -(10+10+18) = -38)$$

Node 9

If fourth object is not included, status of Included objects; (1,1,0,0)

$$c'(7) = -1*10 + 1*10 + 0*12 + 0*18$$

$$= -20$$

$$u(7) = -20 (-\sum \text{fully included objects (that is 1 and 2)} - \sum p_1 + p_2 = -(10+10) = -20)$$

- $\text{upper} = \min\{u(8), u(9)\}$
 $= \min\{-38, -20\}$
 $\text{upper} = -38$
- if $c'(9) = -20 > \text{upper}$, so node 9 is killed
- Nodes 6 and 8 are live nodes with least costs. Regardless of which becomes the next E-node, $c'(E) \geq \text{upper}$ and the search terminates with node 8, the answer node
- At this time the value of upper -38 together with the path 8,7,4,2,1 is printed
 As output

FIFO Branch and Bound

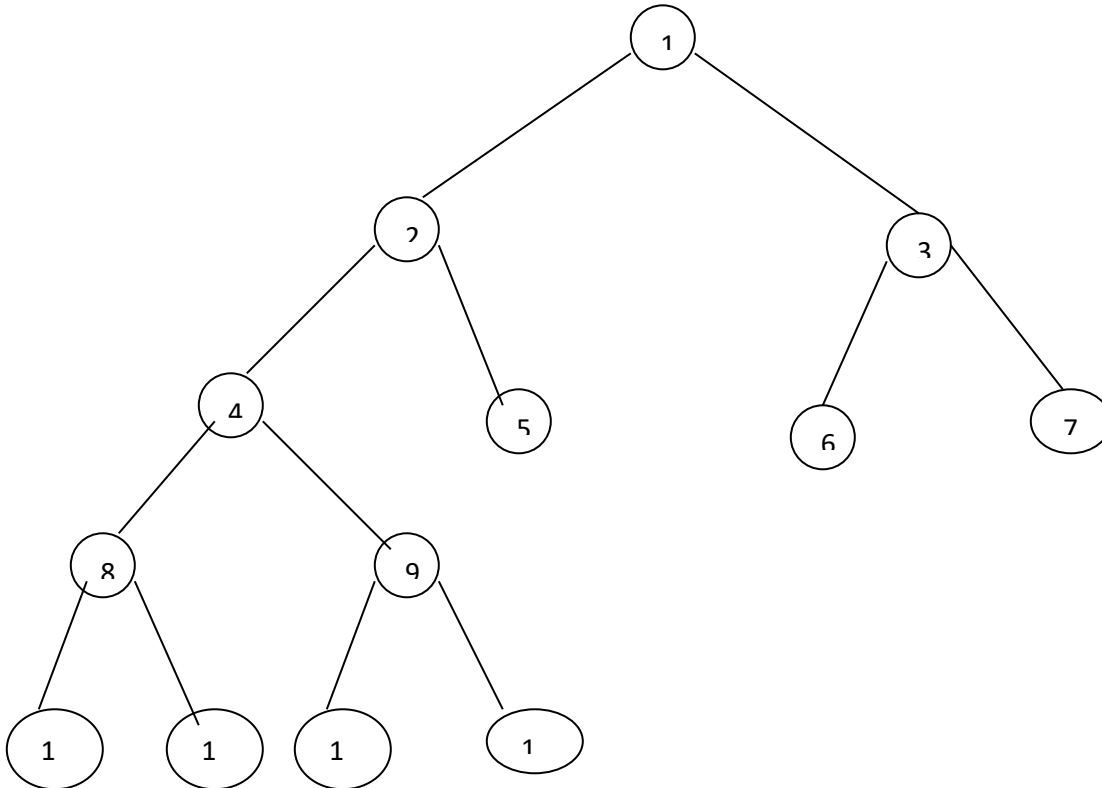
- for any node x in the state space tree, we compute $c'(x)$ and $u(x)$ and upper

- for any node x , if $c'(x) > \text{upper}$ the node x is killed immediately
- initially the $\text{upper} = u(1)$ and then the upper is updated by the node with minimum $u(x)$ value in current level

$u()$ for root node is calculated using the algorithm UBound or $(-\sum \text{fully included objects})$

$u(1) = b(\text{return value of UBound algorithm}) = -32$ (objects 1, 2 and 3 are fully included so $-\sum p_1 + p_2 + p_3 = -(10 + 10 + 12) = -32$)

- Initially the root node, node 1 is the E-node and queue of live nodes are empty
- Since this is not a solution node, upper is initialize to $u(1)$
- Assume that, the children of a node are generated left to right. Node 2 and 3 are generated and added to the queue



- The $c'(2)$, $c'(3)$ and $u(2)$, $u(3)$ are calculated
- $\text{upper} = \min(u(2), u(3))$ that is $\text{upper} = -32$
- Node 2 becomes next E-node. Its children, nodes 4 and 5, are generated and added to the queue
- The $c'(4)$, $c'(5)$ and $u(4)$, $u(5)$ are calculated
- $\text{upper} = \min(u(4), u(5))$ that is $\text{upper} = -32$
- Node 3 becomes next E-node. Its children, nodes 6 and 7, are generated and added to the queue
- The $c'(6)$, $c'(7)$ and $u(6)$, $u(7)$ are calculated
- Node 7 is immediately killed because $c'(7) > \text{upper} (-28 > -32)$
- $\text{upper} = \min(u(6), u(7))$ that is $\text{upper} = -32$
- Node 4 becomes next E-node. Its children, nodes 8 and 9, are generated and added to the queue
- The $c'(8)$, $c'(9)$ and $u(8)$, $u(9)$ are calculated
- $\text{upper} = \min(u(8), u(9))$ that is $\text{upper} = -38$
- nodes 5 and 6 in the queue are the next E-nodes which are not expanded because $c'(5)$ and $c'(6) > \text{upper} (-36 \text{ and } -32 > -38)$
- Node 8 becomes next E-node. Its children, nodes 10 and 11, are generated and added to the queue
- The $c'(10)$, $c'(11)$ and $u(10)$, $u(11)$ are calculated

- Node 10 is infeasible so it is killed. Node 11 has $c'(11) > \text{upper}$ so it is killed
- Node 9 becomes next E-node. Its children, nodes 12 and 13, are generated and added to the queue
- The $c'(12)$, $c'(13)$ and $u(12)$, $u(13)$ are calculated
- $\text{upper} = \min(u(12), u(13))$ that is $\text{upper} = -38$
- node 13 is killed because $c'(13) > \text{upper}$
- The only remaining node is 12. it has no children and search terminates
- The value of upper and the path from node 12 to root is printed as output

5. Explain travelling salesman problem (UQ NOV'12, NOV'10 & APRIL'12)

INTRODUCTION:

It is algorithmic procedures similar to backtracking in which a new branch is chosen and is there (bound there) until new branch is choosing for advancing.

This technique is implemented in the traveling salesman problem [TSP] which are asymmetric ($C_{ij} < C_{ji}$) where this technique is an effective procedure.

STEPS INVOLVED IN THIS PROCEDURE ARE AS FOLLOWS:

STEP 0: Generate cost matrix C [for the given graph g]

STEP 1: [ROW REDUCTION]
For all rows do step 2

STEP: Find least cost in a row and negate it with rest of the elements.

STEP 3: [**COLUMN REDUCTION**]
Use cost matrix- Row reduced one for all columns do STEP 4.

STEP 4: Find least cost in a column and negate it with rest of the elements.

STEP 5: Preserve cost matrix C [which row reduced first and then column reduced] for the i^{th} time.

STEP 6: Enlist all edges (i, j) having cost = 0.

STEP 7: Calculate effective cost of the edges. $\sum (i, j) = \text{least cost in the } i^{\text{th}} \text{ row excluding } (i, j) + \text{least cost in the } j^{\text{th}} \text{ column excluding } (i, j).$

STEP 8: Compare all effective cost and pick up the largest l. If two or more have same cost then arbitrarily choose any one among them.

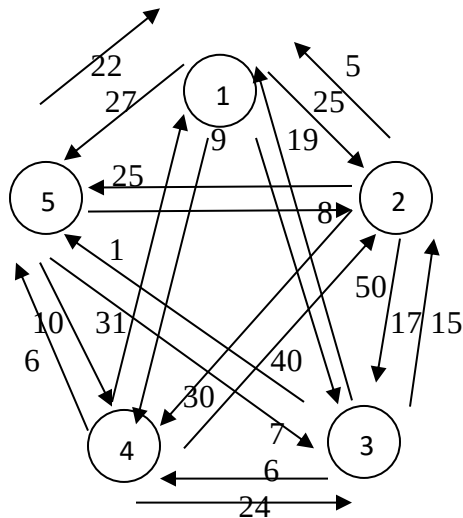
STEP 9: Delete (i, j) means delete i^{th} row and j^{th} column change (j, i) value to infinity. (Used to avoid infinite loop formation) If (i, j) not present, leave it.

STEP 10: Repeat step 1 to step 9 until the resultant cost matrix having order of 2×2 and reduce it. (Both R.R and C.C)

STEP 11: Use preserved cost matrix $C_n, C_{n-1} \dots C_1$
Choose an edge $[i, j]$ having value $=0$, at the first time for a preserved matrix and leave that matrix.

STEP 12: Use result obtained in Step 11 to generate a complete tour.

EXAMPLE: Given graph G



MATRIX:

	1	2	3	4	5
1	α	25	40	31	27
2	5	α	17	30	25
3	19	15	α	6	1
4	9	50	24	α	6
5	22	8	7	10	α

PHASE I

STEP 1: Row Reduction C

C1 [ROW REDUCTION:

	1	2	3	4	5
1	α	0	15	6	2
2	0	α	12	25	20
3	18	14	α	5	0
4	3	44	18	α	0
5	15	1	0	3	α

STEP 3: C1 [Column Reduction]

	1	2	3	4	5
1	α	0	15	3	2
2	0	α	12	25	20
3	18	14	α	2	0
4	3	44	18	α	0
5	15	1	0	3	α

STEP 5:

Preserve the above in C1,

	1	2	3	4	5
1	α	0	15	3	2
2	0	α	12	22	20
3	18	14	α	2	0
4	3	44	18	α	0
5	15	1	0	3	α

STEP 6:

$$L = \{(1,2), (2,1), (3,5), (4,5), (5,3), (5,4)\}$$

STEP 7:

Calculation of effective cost [E.C]

$$(1,2) = 2+1 = 3$$

$$(2,1) = 12+3 = 15$$

$$(3,5) = 2+0 = 2$$

$$(4,5) = 3+0 = 3$$

$$(5,3) = 0+12 = 12$$

$$(5,4) = 0+2 = 2$$

STEP 8:

L having edge (2,1) is the largest.

STEP 9: Delete (2,1) from C1 and make change in it as (1,2) $\rightarrow \alpha$ if exists.

Now Cost Matrix =

	2	3	4	5
1	α	15	3	2
3	14	α	2	0
4	44	18	α	0
5	1	0	0	α

STEP 10: The Cost matrix $\neq 2 \times 2$.
Therefore, go to step 1.

PHASE II:

STEP1: C2(R, R)

	2	3	4	5
1	α	13	1	0
3	14	α	2	0
4	44	18	α	0
5	1	0	0	α

STEP 3: C2 (C, R)

	2	3	4	5
1	α	13	1	0
3	13	α	2	0
4	43	18	α	0
5	0	0	0	α

STEP 5: Preserve the above in C2

C2 =

	2	3	4	5
1	α	13	1	0
3	13	α	2	0
4	43	18	α	0
5	0	0	0	α

STEP 6:

L = {(1,5), (3,5), (4,5), (5,2), (5,3), (5,4)}

STEP 7: calculation of E.C.

$$\begin{aligned}(1,5) &= 1+0 = 1 \\(3,5) &= 2+0 = 2 \\(4,5) &= 18+0 = 18 \\(5,2) &= 0+13 = 13 \\(5,3) &= 0+13 = 13 \\(5,4) &= 0+1 = 1\end{aligned}$$

STEP 8: L having an edge (4,5) is the largest.

STEP 9: Delete (4,5) from C2 and make change in it as (5,4) = α if exists.

Now, cost matrix

	2	3	4
1	α	13	1
3	13	α	2
5	0	0	α

STEP 10: THE cost matrix $\neq 2 \times 2$ hence go to step 1

PHASE III:

STEP 1: C3 (R, R)

	2	3	4
1	α	12	0
3	11	α	0
5	0	0	α

STEP 3: C3 (C, R)

	2	3	4
1	α	12	0
3	11	α	0
5	0	0	α

STEP 5: preserve the above in C2

STEP 6: $L = \{(1,4), (3,4), (5,2), (5,3)\}$

STEP 7: calculation of E.C

$$(1,4) = 12 + 0 = 12$$

$$(3,4) = 11 + 0 = 11$$

$$(5,2) = 0 + 11 = 11$$

$$(5,3) = 0 + 12 = 12$$

STEP 8: Here we are having two edges (1,4) and (5,3) with cost = 12. Hence arbitrarily choose (1,4)

STEP 9: Delete (i,j) $\rightarrow$ (1,4) and make change in it (4,1) = α if exists.

Now cost matrix is

	2	3
2	11	α
3	0	0

STEP 10: We have got 2x2 matrix

C4 (RR)=

	2	3
3	0	α
5	0	0

C4 (C, R) =

	2	3
3	0	α
5	0	0

Therefore, C4 =

	2	3
3	0	α
5	0	0

STEP 11: LIST C1, C2, C3 AND C4

C4	2	3
3	0	α
5	0	0

C3	2	3	4
1	α	12	0
3	11	α	0
5	0	0	α

C2 =

	2	3	4	5
1	α	13	1	0
3	13	α	2	0
4	43	18	α	0
5	0	0	0	α

C1 =

	1	2	3	4	5
1	α	0	15	3	2
2	0	α	12	22	20
3	18	14	α	2	0
4	3	44	18	α	0
5	15	1	0	0	α

STEP 12:

i) Use C4 =

2 3

3	0	α
5	0	0

Pick up an edge (I, j) =0 having least index

Here (3,2) =0

Hence, $T \leftarrow (3,2)$

Use C3 =

	2	3	4
1	α	12	0
3	11	α	0
5	0	0	α

Pick up an edge (i, j) =0 having least index

Here (1,4) =0

Hence, $T \leftarrow (3,2), (1,4)$

Use C2=

	2	3	4	5
1	α	13	1	0
3	13	α	2	0
4	43	18	α	0
5	0	0	0	α

Pick up an edge (i, j) with least cost index.

Here (1,5) $\rightarrow$ not possible because already chosen index i (i=j)

(3,5) $\rightarrow$ not possible as already chosen index.

$(4,5) \rightarrow 0$

Hence, $T \leftarrow (3,2), (1,4), (4,5)$

Use $C1 =$

	1	2	3	4	5
1	α	0	15	3	2
2	0	α	12	22	20
3	18	14	α	2	0
4	3	44	18	α	0
5	15	1	0	0	α

Pick up an edge (i, j) with least index

$(1,2) \rightarrow$ Not possible

$(2,1) \rightarrow$ Choose it

HENCE $T \leftarrow (3,2), (1,4), (4,5), (2,1)$

SOLUTION:

From the above list

3—2—1—4—5

This result now, we have to return to the same city where we started (Here 3).

Final result:

3—2—1—4—5—3

Cost is $15+15+31+6+7=64$

6. Explain NP hard and NP-completeness (UQ APRIL'13, NOV'12 & APRIL'12 ,APRIL/MAY'14/ APRIL 2015)

Apparently some NP problems are “harder” in a relative sense than the other problems. If you can write polynomial algorithm for any one problem in this ‘hard’ group, then it can be shown that every NP-class problem will have a polynomial algorithm. This group of problems is called NP-complete problems.

The secret lies in the fact that they are all “related” to all NP-class problems (!!!) by directed chains of polynomial transformations (explained below).

We will see polynomial transformations first and then come back to the issue of NP-completeness.

Polynomial Transformation

Problem Transformation: some algorithms which take a decision problem X (or rather ANY instance of the problem of type X), and output a corresponding instance of the decision problem of type Y , in such a way that if the input has answer True, then the output (of type Y) is also True and vice versa. [REMINDER: Problem $\equiv$ Input; & Solution $\equiv$ Algorithm that takes any input and produces correct output.]

For example, you can write a problem transformation algorithm from 3-SAT problem to 3D-Matching problem (will see later).

Note that the problem transformations are directed.

When a problem transformation algorithm is polynomial-time we call it a **polynomial transformation**.

Existence of a polynomial transformation algorithm has a great **significance** for the complexity issues.

Suppose you have (1) a poly-transformation A_{xy} exists from a (source) problem X to another (target) problem Y , and (2) Y has a poly algorithm P_y , then you can solve any instance of the source problem X polynomially, by the following method.

Just transform any instance of X into another instance of Y first using A_{xy} , and then use Y 's poly-algorithm P_y . Both of these steps are polynomial, and the output (T/F) from Y 's algorithm is valid for the source instance (of X) as well. Hence, the True/False answer for the original instance of $P_y(A_{xy}(X))$ will be obtained in poly-time. This constitutes an indirect poly-algorithm for X , thus making X also belonging to the P-class.

Note, $|A_{xy}(X)|$ is polynomial with respect to $|X|$.

Cook's theorem.

Cook modeled all NP-problems (an infinite set) to an abstract Turing machine. Then he developed a poly-transformation from this machine (i.e., all NP-class problems) to a particular decision problem, namely, the Boolean Satisfiability (SAT) problem.

Significance of Cook's theorem: if one can find a poly-algorithm for SAT, then by using Cook's poly-transformation one can solve all NP-class problems in poly-time (consequently, P-class = NP-class would be proved).

SAT is the historically first identified NP-hard problem!

Further significance of Cook's theorem: if you find a poly-transformation from SAT to another problem Z , then Z becomes another NP-hard problem. That is, if anyone finds a poly algorithm for Z , then by using your poly-transformation from SAT-to- Z , anyone will be able to solve any SAT problem-instance in poly-time, and hence would be able to solve all NP-class problems in poly-time (by Cook's theorem).

These problems, which have a chain of poly-transformation from SAT, are called **NP-hard problems**.

If an NP-hard problem also belongs to the NP-class it is called an NP-complete problem, and the group of such problems are called **NP-complete problems**.

Pondicherry University Questions

2 Marks

1. Define Branch-and-Bound method. (UQ APRIL'13 & APRIL'12) (Ref.Pg.No.1 Qn.No.1)
2. What are NP- hard and Np-complete problems? (UQ NOV'12) (Ref.Pg.No.1 Qn.No.3)
3. Explain Least Cost (LC) Search (UQ APRIL'13) (Ref.Pg.No.1 Qn.No.7)
4. FIFO Branch and Bound (UQ NOV'12) (Ref.Pg.No. Qn.No.8)
5. Differentiate backtracking and branch-and-bound. Techniques (UQ APR'11) (Ref.Pg.No.1 Qn.No.9)
6. Give the mathematical equation for 0/1 knapsack problem. (UQ APR'11) (Ref.Pg.No.2 Qn.No.18)
7. What is bounding function? (UQ:APRIL/MAY'14) (Ref.Pg.No.3 Qn.No.19)
8. what is non deterministic algorithm (UQ:APRIL/MAY'14)) (Ref.Pg.No.3 Qn.No.20)

11 Marks

1. Explain Branch and bound method (UQ APRIL'12 & APRIL'13, APRIL/MAY'14 Nov'16) (Ref.Pg.No.4 Qn.No.1)
2. Explain Least Cost (LC) Search with 15 puzzle example (UQ APRIL'13) (Ref.Pg.No.6 Qn.No.2)
3. Explain Knapsack problem (UQ NOV'10) (Ref.Pg.No.13 Qn.No.4)
4. Explain travelling salesman problem (UQ NOV'12, NOV'10 & APRIL'12, May/Nov'16) (Ref.Pg.No.21 Qn.No.5)
5. Explain NP hard and NP-completeness (UQ APRIL'13, NOV'12 & APRIL'12 , APRIL/MAY'14/16) (Ref.Pg.No.29 Qn.No.6)